

falco_security_dashboard.json

Enterprise DevSecOps Security Lab

Prepared By	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Document Type	Grafana Dashboard Design and JSON Artifact Review
Environment	Private Ubuntu VM, Kind Kubernetes, Falco, Prometheus, Grafana
Classification	Portfolio / Private Lab Evidence

Document Intent

This professional portfolio artifact describes the design, implementation, validation, and operational usage of runtime detection, alerting, dashboards, or response artifacts created inside the private Enterprise DevSecOps lab.

Coverage Area	Included in this document
Runtime detection	Falco event logic, validation evidence, and triage workflow
Security observability	Prometheus metrics, Grafana dashboarding, alert health checks
Response readiness	Analyst actions, containment commands, retest criteria, evidence handling
Portfolio value	Professional documentation showing detection engineering, SOC process, and cloud-native security maturity

Lab boundary: no external systems were targeted. All attack simulations, detections, and response workflows were performed inside an isolated private VM and Kind Kubernetes environment.

Executive Summary

This document describes the Grafana dashboard JSON artifact used to visualize Falco runtime security metrics in the Enterprise DevSecOps lab. The dashboard is stored as code so it can be reviewed, versioned, imported, and restored like any other infrastructure artifact.

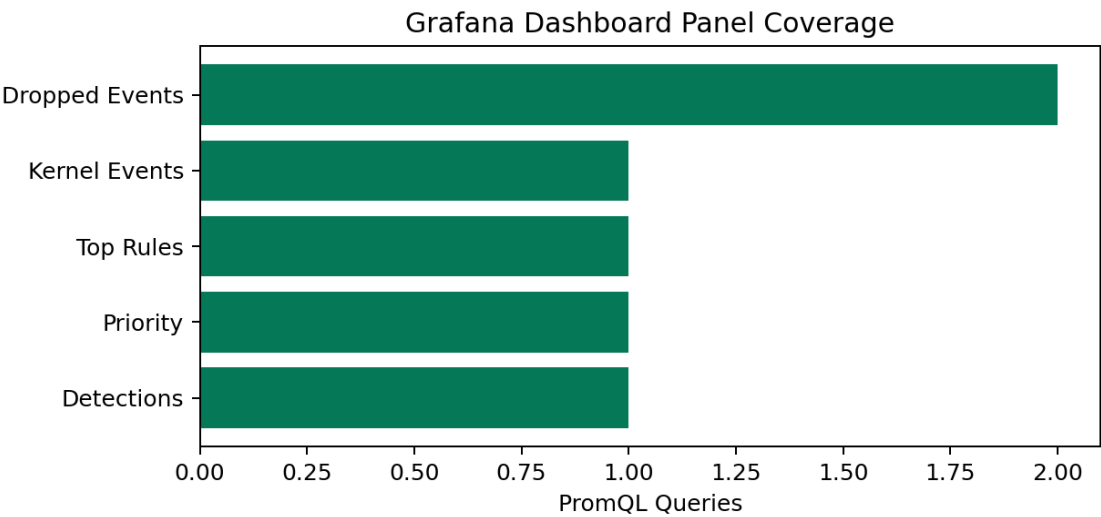
The dashboard focuses on five operational views: recent detection count, events by priority, top triggered rules, kernel event processing rate, and dropped event indicators. This helps an analyst quickly answer whether runtime detections are happening, what severity they carry, which rules are noisy, and whether Falco telemetry is healthy.

Dashboard Metadata

Field	Value
Dashboard title	Falco Runtime Security Dashboard
UID	falco-runtime-security
Refresh interval	10s
Time range	{'from': 'now-1h', 'to': 'now'}
Tags	falco, security, runtime, devsecops
Datasource	Prometheus

Panel Inventory

Panel	Type	PromQL / Purpose
Falco Detections - Last 10m	stat	sum(increase(falcosecurity_falco_rules_matches_total[10m]))
Falco Events by Priority	timeseries	sum by (priority) (increase(falcosecurity_falco_rules_matches_total[30m]))
Top Falco Rules Triggered - Last 1h	table	topk(10, sum by (rule_name) (increase(falcosecurity_falco_rules_matches_total[1h])))
Falco Kernel Event Processing Rate	timeseries	sum(rate(falcosecurity_scap_n_evts_total[5m]))
Falco Dropped Events / Output Drops	timeseries	sum(increase(falcosecurity_scap_n_drops_total[10m])) sum(increase(falcosecurity_falco_outputs_queue_num_drops_total[10m]))



Dashboard panel coverage and PromQL query count.

Dashboard Design Rationale

Design Choice	Reason
Stat panel for last 10 minutes	Provides fast executive visibility into whether detections occurred recently
Priority trend panel	Helps separate low-noise warnings from critical alerts
Top rule table	Identifies noisy or high-risk rules that need tuning or response
Kernel event processing rate	Confirms Falco is receiving runtime telemetry
Dropped event panels	Detects monitoring integrity issues where alerts may be missed

Import and Validation Procedure

```
# Store dashboard as code
monitoring/dashboards/falco-security-dashboard.json

# Optional: load as Grafana dashboard ConfigMap
kubectl -n monitoring create configmap falco-security-dashboard
--from-file=falco-security-dashboard.json=monitoring/dashboards/falco-security-dashboard.json
# Validate Prometheus has Falco metrics
curl -s "http://localhost:9090/api/v1/label/__name__/values" | grep falcosecurity
```

Full Dashboard JSON Artifact

The following JSON is the version-controlled dashboard artifact used by the lab. It can be imported into Grafana or converted into a ConfigMap for automated provisioning.

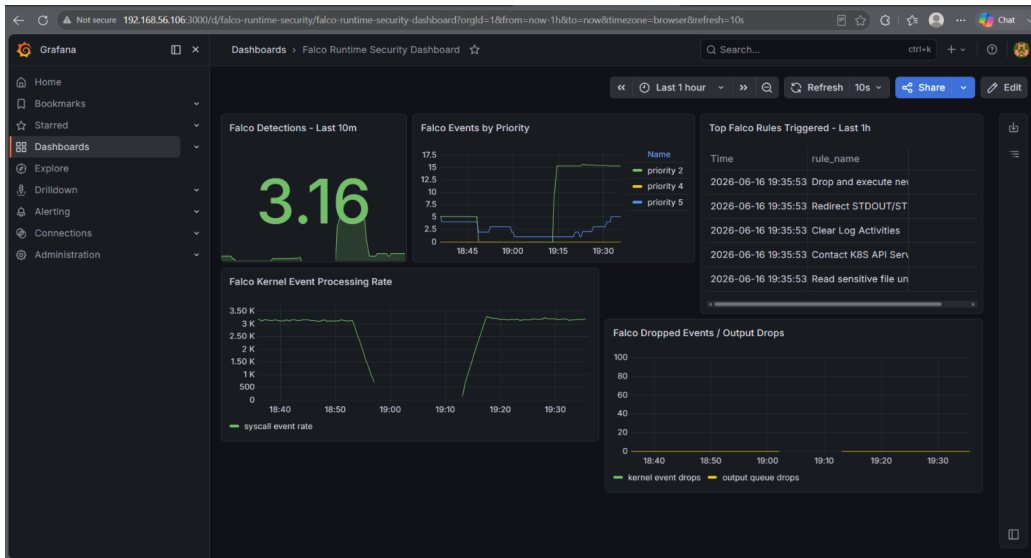
```

{
  "annotations": {
    "list": []
  },
  "editable": true,
  "fiscalYearStartMonth": 0,
  "graphTooltip": 0,
  "id": null,
  "links": [],
  "liveNow": false,
  "panels": [
    {
      "datasource": {
        "type": "prometheus",
        "uid": "prometheus"
      },
      "fieldConfig": {
        "defaults": {
          "unit": "short"
        },
        "overrides": []
      },
      "gridPos": {
        "h": 6,
        "w": 6,
        "x": 0,
        "y": 0
      },
      "id": 1,
      "options": {
        "colorMode": "value",
        "graphMode": "area",
        "justifyMode": "auto",
        "orientation": "auto",
        "reduceOptions": {
          "calcs": [
            "lastNotNull"
          ],
          "fields": "",
          "values": false
        },
        "textMode": "auto"
      },
      "targets": [
        {
          "expr": "sum(increase(falcosecurity_falco_rules_matches_total[10m]))",
          "legendFormat": "Falco detections",
          "refId": "A"
        }
      ],
      "title": "Falco Detections - Last 10m",
      "type": "stat"
    },
    {
      "datasource": {
        ... truncated for report readability (165 additional lines in repository artifact) ...

```

Operational Usage

- Use the last-10-minute detection stat for rapid SOC handoff and demonstration screenshots.
- Use the priority panel to identify critical/high runtime events during attack simulations.
- Use the top-rule table for tuning false positives and documenting detection coverage.
- Use drop metrics to confirm telemetry quality before relying on detection counts.



Evidence: Custom Falco Runtime Security Dashboard imported into Grafana.

Change Control

Change Type	Review Requirement
PromQL change	Validate query in Prometheus before commit
Panel removal	Confirm replacement coverage exists
Refresh interval change	Check Prometheus and Grafana resource impact
Datasource UID change	Verify dashboard loads in clean lab deployment

References

Falco rules basic elements - <https://falco.org/docs/concepts/rules/basic-elements/>

Falco supported fields for rule conditions and outputs - <https://falco.org/docs/reference/rules/supported-fields/>

Falco metrics - <https://falco.org/docs/concepts/metrics/>

Falco alert forwarding and JSON outputs - <https://falco.org/docs/concepts/outputs/forwarding/>

Prometheus alerting rules - https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

Grafana dashboard JSON model -

<https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/view-dashboard-json-model/>

Grafana dashboard import workflow - <https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/import-dashboards/>